

ign1t10n: macOS Installer Specification

One install: a multi-validator F1R3Node-Rust shard, its observer, and the F1R3Gaze browser

Version 0.3 (draft for review)

Lucius Gregory Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

30 September 2026

Abstract

This specification defines ign1t10n, an installer for macOS that leaves a person, a few minutes after double-clicking one file, with a working F1R3Node-Rust shard on their own machine: two validators by default, a read-only observer joined to them, and the F1R3Gaze browser configured to deploy to the shard and read from it, paying from a wallet that the shard's genesis has funded. A configuration panel lets the user change the number of validators, from two up to ten, either by growing or shrinking the running shard (new validators join, are funded and bond at the next epoch; departing ones withdraw first) or by starting a new shard of the chosen size. The specification fixes the artifact (a signed, notarised product archive carrying F1R3Gaze and the ign1t10n menu-bar application, which owns the shard), first-run provisioning, the supervision of every node by one per-user launch agent, the configuration written to each program, the resizing procedure, the security model, upgrade, reset and uninstall, the build and release pipeline, and acceptance tests. It is grounded in a review of F1R3Node-Rust at fce422a, F1R3Gaze at f6ee26a and CampF1R3 at b46e16c (with F1R3Gaze's pinned CampF1R3 revision ea2d323), and of the F1R3Gaze architecture proposal and implementation specification in publications/f1r3gaze.

Version 0.3 replaces the single standalone validator of version 0.2 with the multi-validator topology the node repository already runs and tests (docker/shard.yml), generalised to N validators on loopback.

Contents

1	Scope and conventions	4
1.1	What the installer delivers	4
1.2	Non-goals for this version	4
1.3	Pinned baselines	4
1.4	Conventions	5
2	Findings from the code review	5
3	Architecture	7
3.1	Components	8
3.2	Principles	8
4	The distribution artifact	8

5	ign1t10n	9
5.1	Bundle layout	9
5.2	One binary, three roles	9
5.3	The launch agent	10
5.4	The state directory	10
5.5	Keys and Keychain items	11
6	Provisioning	11
6.1	S0: preflight	11
6.2	S1: ports	11
6.3	S2: the paying wallet	11
6.4	S3: keys	12
6.5	S4: genesis files	12
6.6	S5: node configurations	12
6.7	S6: register the agent	12
6.8	S7: genesis	12
6.9	S8: configure F1R3Gaze	13
6.10	S9: open the browser	13
6.11	Budgets	13
7	Supervision	13
7.1	Start	13
7.2	Health	14
7.3	Restarts	14
7.4	Stop	14
7.5	Port conflicts at start	14
7.6	Sleep and wake	15
8	Resizing the shard	15
8.1	In place (default)	15
8.2	As a new shard	16
8.3	Invariants	16
9	User interface	16
9.1	Menu bar	16
9.2	The configuration panel	16
9.3	First-run window	17
10	Integration with F1R3Gaze	17
11	Required changes to F1R3Node-Rust	18
12	Security model	18
13	Upgrade, reset and uninstall	18
13.1	Upgrade	18
13.2	Reset	19
13.3	Uninstall	19

14 Build and release pipeline	19
15 Testing and acceptance	20
16 Work packages and schedule	21
17 Obligations and decisions	22
A Configuration templates	22
B The manifest	24

1 Scope and conventions

1.1 What the installer delivers

After a successful install on a supported Mac, with no further action beyond approving a background item in System Settings when macOS asks:

1. `/Applications/F1R3Gaze.app` and `/Applications/ign1t10n.app` are installed, signed and notarised.
2. A per-user launch agent, `io.f1r3fly.ign1t10n.supervisor`, is registered and running. It supervises the shard's nodes:
 - a *bootstrap* node, which coordinated the genesis ceremony and is every other node's rendezvous; it is not bonded and never proposes;
 - N *validators*, bonded at genesis, which propose and finalise blocks ($N = 2$ unless the user has chosen otherwise, $2 \leq N \leq 10$);
 - one *observer*, a read-only node that serves every read the browser makes.
3. Every key was generated on this machine at install time. The shard's genesis funds the user's active F1R3Gaze wallet and a local faucet.
4. F1R3Gaze deploys to validator 1, reads from the observer, and is open in a window.
5. Every listening socket of every node is bound to the loopback interface.
6. The shard survives logout, sleep and reboot, and comes back with its chain intact, at the size last chosen.

Throughout, “ N nodes” in the user interface means N validators; the bootstrap and the observer are always present and are shown separately. With the default, five `f1r3node` processes run: bootstrap, two validators, observer.

1.2 Non-goals for this version

More than ten validators; joining a remote or public shard; exposing the local shard to the network; stake other than equal stakes; Linux and Windows (the F1R3Gaze release already ships those installers, but the shard bundle here is macOS-only); bundling the Embers service; Docker.

1.3 Pinned baselines

Source	Revision	Notes
F1R3Node-Rust	master fce422a	@ crate node v0.4.46; toolchain nightly-2026-02-09; topology reference docker/shard.yml, docker/conf/default.conf, docker/conf/bootstrap.conf; no macOS CI job exists
F1R3Gaze	main f6ee26a	@ Rust 1.91, edition 2024; P4 release workflow producing a universal, notarised DMG; crates gaze-shard and gaze-wallet reused by ign1t10n
CampF1R3	main b46e16c	@ consumed only through F1R3Gaze, which pins branch gaze/map-remainders @ ea2d323
Publications	f1r3gaze/	architecture proposal v0.2; implementation specification v0.1 (work package K1 names installers in phase P4)
Publications	fuzzyware/	background only; nothing in the installer depends on the graded resolver, which F1R3Gaze does not yet ship

1.4 Conventions

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY carry their RFC 2119 meanings. “The state directory” is `~/Library/Application Support/F1R3FLY/ign1t10n`. “The profile” is F1R3Gaze’s profile directory, `~/Library/Application Support/F1R3Gaze` unless `F1R3GAZE_PROFILE` is set. `ign1t10n` names the product: the installer archive, the menu-bar application that manages the shard, its binary, its launch agent and its repository all carry the one name. “Slot *k*” is validator *k*’s place in the port plan (§6.2); validator 1 is never removed. Ports are written for the default allocation.

2 Findings from the code review

These are the facts the design rests on. Each cites where it was established.

Finding 1 (The observer is not optional). Every endpoint that runs an exploratory deploy is served only by read-only nodes; a validator answers `400 readonly_node_required`. The list (`docs/node/api-reference.md`) includes `POST /api/explore-deploy`, `POST /api/estimate-cost`, `GET /api/registry/{uri}` and `GET /api/balance/{address}`. F1R3Gaze’s shard bridge uses exactly these: it resolves sites and reads registry values at observers (`gaze-shard/src/bridge.rs`, `graded`), quotes the cost of a deploy by `estimate_cost` on the first observer, subscribes to `/ws/events` on the first observer, and sends deploys to the validator.

Finding 2 (The shipped topologies). The repository runs two topologies. *Standalone* (`docker/standalone.yml`): one bonded node that is its own ceremony master, with no peers. *Shard* (`docker/shard.yml`): a bootstrap node started with `--ceremony-master-mode`, `--required-signatures`, and `--heartbeat-disabled`, bootstrapping from itself and absent from `bonds.txt` (“The bootstrap node is NOT a validator”, `docker/conf/bootstrap.conf`); genesis validators started with `--genesis-validator` and `--bootstrap` pointing at it; and an observer bootstrapping from it. Nothing in the repository pairs a standalone node with an observer, and that pairing proved awkward in practice; the shard topology is the one that is tested (CI smoke, soak runs). This specification adopts the shard topology, generalised from three validators to *N*.

Finding 3 (Same host, distinct ports). In `shard.yml` every node listens on the same six container ports and is told apart by its Docker hostname; the host publishes them at 40400, 40410, 40420, 40430 (and 40440 for `validator4.yml`) and 40450 for the observer (`docker/README.md`, “Port Mapping”). On one Mac without Docker every node must advertise `127.0.0.1` and differ by port. The node supports per-node ports through configuration and `--host`; that a set of nodes on one loopback address discover and gossip correctly is expected but untested (Obligation 1).

Finding 4 (The browser’s default validator would be the bootstrap). `ShardConfig::default()` in F1R3Gaze is validator `http://localhost:40403`, observers [`http://localhost:40453`], shard id `root`, quorum 2. In the shard topology port 40403 is the *bootstrap*, which is not bonded. A node that cannot propose accepts a deploy and strands it, because deploy queues are node-local (`docs/node/joining-a-network.md`, and `/api/pending-deploys` in the API reference). `ign1t10n` therefore always writes F1R3Gaze’s validator explicitly, to validator 1. With one observer the bridge returns the *node* rung and never consults the quorum, so `ign1t10n` writes `quorum = 1`.

Finding 5 (The shard can be resized while running). The join guide (`docs/node/joining-a-network.md`) specifies how a node that was not at genesis joins (last-finalised-state fetch from `--bootstrap`) and becomes a validator: start it already carrying its validator key; fund

its vault by transfer from a funded key; submit a bond deploy *to a bonded validator*, signed by the joiner's own key (`rho-lang/examples/bond/bond.rho`, calling `PoS!("bond", deployerId, amount, ret)`); the bond takes effect at the next epoch boundary, observable at `GET /api/bond-status/{pk}`. Leaving is the mirror image: `PoS!("withdraw", deployerId, ret)` (`casper/src/main/resources/PoS.rhox`, line 370) takes the validator out of the active set at the next boundary and pays out stake after the quarantine. The shard configuration sets `epoch-length = 10` and `quarantine-length = 10` (`docker/conf/default.conf`), so both take seconds to a few minutes locally, not the thousands of blocks of the node defaults. The guide warns: "Only bond a validator you intend to keep running. A bonded validator that does not propose is counted in consensus while contributing nothing to it."

Finding 6 (Deploys can be signed outside the node). `F1R3Gaze`'s `gaze-shard` produces deploys byte-for-byte as the node verifies them (tested against `prost`), and `gaze-wallet` renders the vault transfer template (`render_transfer`) and derives `F1R3Cap` addresses. `ign1t10n` reuses both crates, pinned to the `F1R3Gaze` release it ships, for faucet transfers, bond and withdraw deploys, so it needs neither the node's deploy subcommand nor an external client.

Finding 7 (The shipped development keys are public). `docker/.env.example` (and the committed `docker/.env`) carry the private keys of the bootstrap, four validators and the standalone node; their public keys are the entries of `docker/genesis/bonds.txt` and `standalone-bonds.txt`, and the vaults they control hold the genesis supply. Any install reusing them would give every reader of the repository the validator keys and the funds.

Finding 8 (How a key can reach the node). `--validator-private-key` puts the key in the process's argument vector, which any local user can read with `ps`; `shard.yml` uses it, which is harmless in a container but not on a shared Mac. `--validator-private-key-path` takes an encrypted PEM, decrypted with `OpenSSL` (`Secp256k1::parse_pem_file`); the password comes from `F1R3NODE_VALIDATOR_PASSWORD`, or else from an interactive prompt (`node/src/rust/effects/console_io.rs`).

Finding 9 (Everything listens on all interfaces). `api-server.host` defaults to `0.0.0.0` and is configurable. The protocol transport binds `0.0.0.0` unconditionally (line 600 of `grpc_transport_receiver.rs` in `comm/src/rust/transport`). With up to twelve nodes, that would be up to seventy-two sockets, including internal gRPC (`propose`) and admin APIs, open to whatever network the laptop joins.

Finding 10 (Native linking on macOS). The crypto crate depends on `openssl 0.10` without the vendored feature, so a macOS build links Homebrew's `libssl/libcrypto` dynamically. `LMDB` is compiled from source through `heed`; `aws-lc-sys`, `zstd-sys` and `bzip2-sys` link statically. `protoc` is build-time only. No workflow in the repository builds for macOS.

Finding 11 (`F1R3Gaze` already packages itself for macOS). `packaging/macos/package.sh` builds a universal bundle, `io.f1r3fly.f1r3gaze`, minimum macOS 11, signed with the hardened runtime and *empty* entitlements, in a signed, notarised, stapled DMG. Wallet keys live in the Keychain under service `F1R3Gaze`. `settings.conf` is `key = value` and the last assignment of a key wins (`gaze-shell/src/profile.rs`); if the file is absent at first load `F1R3Gaze` writes a commented template.

Finding 12 (The browser's wallet can be funded at genesis). `F1R3Gaze` wallet addresses are `F1R3Cap` addresses, the same base58 form as the vault addresses in the node's `wallets.txt`. The active wallet signs, and pays for, every deploy the browser makes. Balances and transfers go through Embers only; with `embers_api` unset the wallet panel reports that balances need Embers.

Finding 13 (Lifecycle facts). The node handles SIGTERM and SIGINT. A joined node restarted on the same data directory reconnects in about a second; its identity is the TLS certificate in the data directory, so a data directory must never be deleted while peers remember it. The genesis ceremony in `default.conf` checks for signatures every 10 s (`approve-interval`) and waits at most a minute (`approve-duration`).

3 Architecture

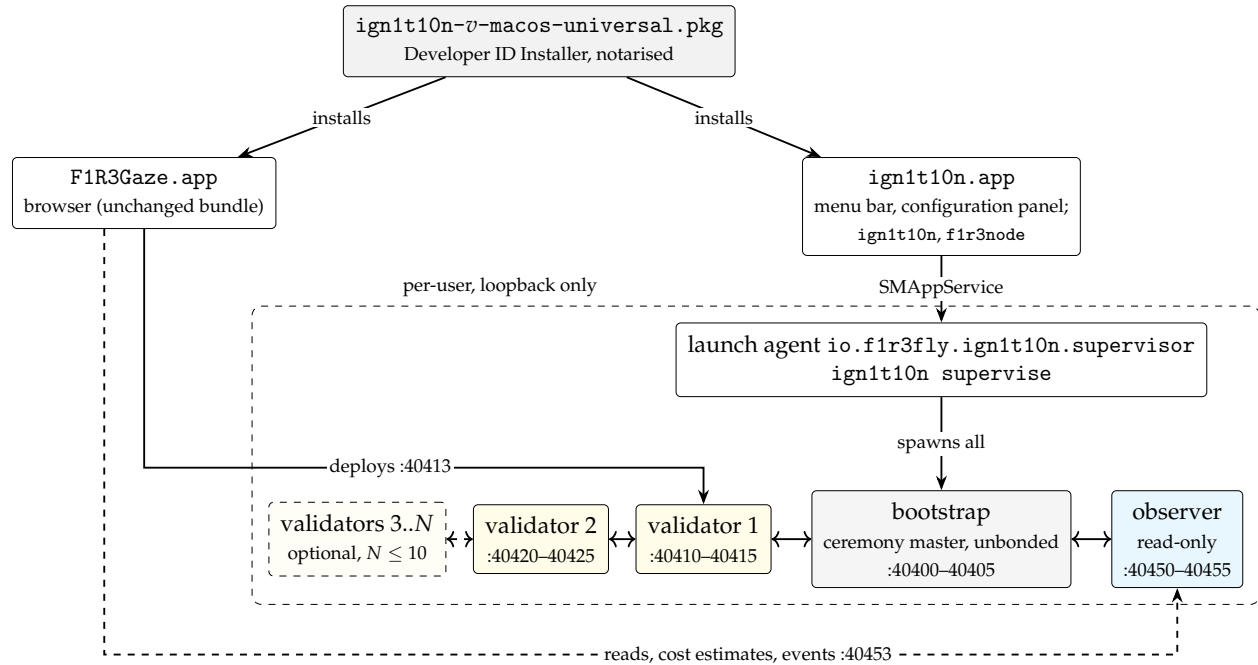


Figure 1: What the installer lays down and what runs afterwards, for the default $N = 2$. Arrows between nodes indicate the peer protocol; every node discovers every other through the bootstrap.

3.1 Components

Component	Role
F1R3Gaze.app	The F1R3Gaze release bundle, byte for byte.
ign1t10n.app	Menu-bar application with a configuration panel; provisions, starts, stops, resizes, resets and reports on the local shard. Contains ign1t10n (one Rust binary with three roles) and f1r3node (the F1R3Node-Rust node binary, universal).
launch agent	io.f1r3fly.ign1t10n.supervisor; runs ign1t10n supervise, which owns every node process.
bootstrap	f1r3node run as ceremony master with --required-signatures N_0 (the genesis size), heartbeat disabled; not in bonds.txt; every other node's --bootstrap.
validators 1..N	f1r3node run with a validator key; genesis validators also --genesis-validator. Validator 1 receives F1R3Gaze's deploys and ign1t10n's administrative deploys.
observer	f1r3node run with no validator key; serves every read.
faucet	A key held by ign1t10n, never by a node, whose vault holds the undistributed genesis supply; funds joining validators and "Fund a wallet".

3.2 Principles

Principle 1 (Per-user, unprivileged at run time). Administrator rights are used once, by Installer.app, to copy two bundles into /Applications. Everything after that runs as the logged-in user.

Principle 2 (Loopback only). No socket of any node listens on a non-loopback address.

Principle 3 (Fresh secrets). Every key and password is generated on the user's machine. None is shipped.

Principle 4 (One owner for the shard). A single supervisor starts, orders, monitors, resizes and stops every node. launchd keeps the supervisor alive; the supervisor keeps the nodes alive. Per-node launch agents were rejected: launchd has no dependency ordering, and resizing must be one transaction over several nodes.

Principle 5 (A bonded validator runs). A validator in the active set is never stopped on its own. Removing a validator means withdrawing it first and stopping it only after it has left the active set (Finding 5). The shard as a whole may be stopped; all validators then stop together.

Principle 6 (Touch only what was created). ign1t10n never deletes or overwrites data it did not create: never a F1R3Gaze wallet, never F1R3Gaze settings outside its own managed block, never ~/ .rnode.

Principle 7 (Idempotent, resumable operations). Provisioning and every resize are sequences of steps whose completion is recorded in shard.toml. Interrupted at any point, each resumes at its first incomplete step.

4 The distribution artifact

Requirement 1 (Product archive). The release artifact **MUST** be a flat product archive, ign1t10n-v-macos-universal.pkg, built with productbuild from two component packages (io.f1r3fly.f1r3gaze.pkg, io.f1r3fly.ign1t10n.pkg), signed with a *Developer ID Installer*

certificate, notarised, and stapled. Each component MUST install its bundle to /Applications with bundle-version checking on, so an installed newer F1R3Gaze is not downgraded.

A DMG was rejected because drag-and-drop cannot launch anything, and the brief asks that the install end with the shard and the browser running.

Requirement 2 (The post-install hook does one thing). The `postinstall` script of the component package `io.f1r3fly.ign1t10n.pkg` MUST do nothing but launch `ign1t10n` in the console user's session:

```
#!/bin/bash
user=$(stat -f %Su /dev/console)
[ "$user" = root ] || [ -z "$user" ] && exit 0      # no GUI session: first launch will
do it
uid=$(id -u "$user")
launchctl asuser "$uid" sudo -u "$user" \
  open -g -a /Applications/ign1t10n.app --args --first-run
exit 0
```

It MUST NOT create files in the user's home, register agents or generate keys. It MUST exit 0 even if the launch fails, so a headless install succeeds and provisioning happens at first launch.

Requirement 3 (Platforms). The artifact MUST run on Apple silicon and SHOULD run on Intel (Decision 2); every Mach-O is universal. The minimum macOS is 13.0, the first release with `SMAccessService` (Decision 3).

Requirement 4 (No dynamic dependencies outside the system). For every Mach-O in the archive, `otool -L` MUST list only paths under `/usr/lib` and `/System/Library`.

5 ign1t10n

5.1 Bundle layout

```
ign1t10n.app/Contents/
  Info.plist                io.f1r3fly.ign1t10n, LSUIElement = true, min 13.0
  MacOS/ign1t10n            menu bar (default), `supervise`, `ctl`
  Helpers/f1r3node          f1r3node-rust `node` v0.4.46, universal
  Library/LaunchAgents/io.f1r3fly.ign1t10n.supervisor.plist
  Resources/
    ign1t10n.icns, StatusTemplate.pdf    F1R3FLY.io brand kit (firefly bug)
    templates/common.conf.in, bootstrap.conf.in, validator.conf.in,
      observer.conf.in, gaze-settings.in, bond.rho, withdraw.rho
    compat.toml                node versions whose data directories are compatible
    uninstall.sh
```

5.2 One binary, three roles

`ign1t10n` is written in Rust (stable 1.91, matching F1R3Gaze), using `objc2` bindings for AppKit, ServiceManagement and Security; `k256` and `pkcs8` for keys; `gaze-shard` and `gaze-wallet` for deploys, transfers and addresses (Finding 6); `toml` and `serde` for the manifest; a blocking HTTP client for the node APIs.

ign1t10n (no arguments)

The menu-bar application and the configuration panel. On first run it opens the first-run window and drives provisioning. It never runs a node itself; it talks to the supervisor over the control socket.

ign1t10n supervise

The launch agent's program. Owns every node process and executes resizes (§7, §8).

ign1t10n ctl *verb*

A command-line client of the control socket: `status [--json]`, `start`, `stop`, `restart`, `resize N [--new-shard]`, `fund address amount`, `logs node`, `env`, `reset`, `uninstall`. It is what the tests drive.

The control socket is `run/control.sock` in the state directory, a Unix-domain socket in a 0700 directory carrying one JSON object per line. The supervisor MUST check the peer's user id (`getpeereid`) and refuse any other user.

5.3 The launch agent

```
<dict>
  <key>Label</key>          <string>io.f1r3fly.ign1t10n.supervisor</string>
  <key>BundleProgram</key>  <string>Contents/MacOS/ign1t10n</string>
  <key>ProgramArguments</key> <array><string>ign1t10n</string><string>supervise</string></array>
  <key>RunAtLoad</key>      <true/>
  <key>KeepAlive</key>      <dict><key>SuccessfulExit</key><false/></dict>
  <key>ProcessType</key>    <string>Standard</string>
  <key>ThrottleInterval</key> <integer>10</integer>
  <key>ExitTimeOut</key>    <integer>180</integer>
  <key>AssociatedBundleIdentifiers</key> <string>io.f1r3fly.ign1t10n</string>
</dict>
```

ProcessType is Standard, because launchd throttles CPU and I/O of background jobs. ExitTimeOut covers the ordered shutdown of up to twelve nodes (§7.4).

5.4 The state directory

```
~/Library/Application Support/F1R3FLY/ign1t10n/    0700
shard.toml                                          the manifest (Appendix B)
keys/bootstrap.pem, validator-1.pem ... validator-10.pem, faucet.pem  0600
genesis/bonds.txt, wallets.txt
conf/common.conf, bootstrap.conf, validator-<k>.conf, observer.conf
nodes/bootstrap/, nodes/validator-<k>/, nodes/observer/  data directories
run/control.sock, supervisor.pid
archive/<timestamp>/                               retired validators and previous shards
~/Library/Logs/F1R3FLY/ign1t10n/
ign1t10n.log                                       supervisor and UI; 10 MB x 5
<node>.stdout.log                                one per node; panics and pre-logger output; 10 MB x 3
```

5.5 Keys and Keychain items

Every key is a secp256k1 key written as an ENCRYPTED PRIVATE KEY PEM (PKCS#8, PBES2, PBKDF2-HMAC-SHA256 with 600 000 iterations, AES-256-CBC), which OpenSSL decrypts and `parse_pem_file` accepts. Each PEM has its own random 32-byte password in a Keychain item of service `io.f1r3fly.ign1t10n`, account `pem.name` (`pem.bootstrap`, `pem.validator-3`, `pem.faucet`, ...). The items' access control list is the application's designated requirement, so the menu bar, supervisor and `ctl` (one binary) read them without a prompt and other programs cannot. A node receives its password only in its own environment; the faucet key is decrypted only inside `ign1t10n` when it signs.

6 Provisioning

Provisioning runs in the menu-bar process, as the user, when `shard.toml` is absent or records an incomplete stage. It creates a shard of N_0 validators, N_0 being 2 unless the first-run window's "Advanced" disclosure was used to choose another value in $[2, 10]$.

6.1 S0: preflight

Checks: `macOS` ≥ 13 ; `F1R3Gaze.app` present with bundle identifier `io.f1r3fly.f1r3gaze` and a valid signature from the same team as `ign1t10n`; free disk of at least $D_{\min}(N_0)$ and physical memory of at least $M_{\min}(N_0)$ (§6.11); the state directory creatable.

6.2 S1: ports

Each node needs six ports (protocol, gRPC external, gRPC internal, HTTP, discovery, admin) at offsets $+0 \dots +5$ from a base. The port plan follows the node's own convention (Finding 3) and keeps the observer where F1R3Gaze expects it:

Node	base	HTTP	note
bootstrap	40400	40403	
validator 1–4	40410, 40420, 40430, 40440	40413, ..., 40443	as <code>shard.yml</code> , <code>validator4.yml</code>
observer	40450	40453	F1R3Gaze's default observer
validator 5–10	40460, 40470, ..., 40510	40463, ..., 40513	skips the observer's decade

A base is usable if its six ports bind on `127.0.0.1`. Each node's base is allocated independently: the plan's base if usable, otherwise the first usable base in `41400, 41410, ..., 49990`. Allocated bases are recorded per node in `shard.toml` and reused on every start; slots not yet in use are not reserved.

6.3 S2: the paying wallet

Using work package G1: `f1r3gaze wallet active --porcelain`; if there is no active wallet, `f1r3gaze wallet new "Local shard" --porcelain`. The address is recorded as `wallet.funded`. `ign1t10n` never sees the key.

6.4 S3: keys

Generate keys for the bootstrap (which signs as ceremony master but is not bonded), for validators $1..N_0$, and for the faucet, each with its PEM and Keychain item. Record every public key and F1R3Cap address. A unit test MUST round-trip a PEM written this way through `parse_pem_file` from the pinned F1R3Node-Rust.

6.5 S4: genesis files

```
# genesis/bonds.txt: <public key> <stake>, one line per genesis validator (not the
bootstrap)
04a1...9e 1000
04c7...31 1000
# genesis/wallets.txt: <F1R3Cap address>,<balance in dust> (8 decimals)
<wallet.funded>,1000000000000000 # 1 000 000 F1R3: the browser's wallet
<validator 1 address>,1000000000000 # 1 000 F1R3 each: fees for its own withdraw
<validator 2 address>,1000000000000
<faucet address>,4980000000000000 # the rest of the dev supply
```

The amounts are Decision 4; the faucet's balance leaves room for eight further validators' stakes and fees.

6.6 S5: node configurations

Render `conf/common.conf` (network id `ign1t10n-8` random hex digits, fresh at each genesis; shard name `root`; the consensus, genesis and epoch parameters of `docker/conf/default.conf` with `required-signatures = N0`; logging; loopback binding) and one small file per node that includes it and sets that node's ports, data directory and role (Appendix A).

6.7 S6: register the agent

`SMAppService.agent(plistName: "io.f1r3fly.ign1t10n.supervisor.plist").register();` if the status is `requiresApproval`, the first-run window explains and offers `SMAppService.openSystemSettingsLoginItems()`, then waits for enabled.

6.8 S7: genesis

The supervisor performs the genesis start:

1. Start the bootstrap; wait for its `/api/status` to answer; record its node id from the address field.
2. Start validators $1..N_0$ with `--genesis-validator` and `--bootstrap` at the bootstrap.
3. Wait until every validator's and the bootstrap's `/api/ready` answers 200 (the ceremony has gathered N_0 signatures and every node has the approved block). Record the genesis block hash.
4. Start the observer bootstrapping from the bootstrap; wait for its `/api/ready`.
5. Verify each genesis validator with `GET /api/bond-status/{pk}`.

6.9 S8: configure F1R3Gaze

With work package G3, write `settings.d/50-ign1t10n.conf`; until G3 lands, maintain a managed block at the end of `settings.conf`, which governs the keys it sets because the last assignment wins (Finding 11):

```
# >>> managed by ign1t10n: do not edit inside this block >>>
validator = http://127.0.0.1:40413
observers = http://127.0.0.1:40453
shard_id = root
quorum = 1
phlo_price = 1
# <<< managed by ign1t10n <<<
```

If the profile already names a non-loopback validator or observer, the first-run window asks before pointing F1R3Gaze at the local shard; the choice is a toggle in the configuration panel afterwards.

6.10 S9: open the browser

open -b io.f1r3fly.f1r3gaze --args gaze://newtab, only after the observer is ready. The first-run window closes with a summary: shard id, validator count, the funded wallet and its balance, and where the logs are.

6.11 Budgets

Targets on an M-series Mac with $N_0 = 2$, measured by the acceptance suite: S0–S6 under 10 s of machine time (approval excluded); S7 steps 1–3 (genesis) under 120 s, given the 10 s approve interval; observer join under 180 s; whole first run under five minutes.

$M_{\min}(N)$ and $D_{\min}(N)$ are $(N + 2)$ times the per-node resident memory and disk measured by Obligation 3 at the pinned node version, plus a margin; they are compiled into `ign1t10n` from that measurement, not guessed. The configuration panel uses the same figures (§9.2).

7 Supervision

7.1 Start

1. Read `shard.toml`; refuse to start if the installed `f1r3node` is incompatible with the recorded storage format (§13.1); resume any incomplete resize (§8) after the start.
2. Probe every recorded port on `127.0.0.1`; on conflict go to §7.5.
3. Start the bootstrap; wait for `/api/ready` (120 s).
4. Start every validator whose recorded state is active, joining, bonding or retiring, in parallel; wait for each `/api/ready` (120 s).
5. Start the observer; wait for `/api/ready` (120 s).
6. Report Running.

Each node runs with an environment reduced to `HOME`, `PATH`, `TMPDIR`, `LANG`, `OPENAI_ENABLED=false`, `OLLAMA_ENABLED=false`, and, for nodes with a key, `F1R3NODE_VALIDATOR_PASSWORD`. Stdout and stderr go to the node's `.stdout.log`. Argument vectors:

```
# bootstrap
f1r3node run --config-file "$STATE/conf/bootstrap.conf" \
  --bootstrap "rnode://$BOOT_ID@127.0.0.1?protocol=40400&discovery=40404" \
```

```

--validator-private-key-path "$STATE/keys/bootstrap.pem" \
--required-signatures $N0 --ceremony-master-mode --heartbeat-disabled \
--host 127.0.0.1 --no-upnp --allow-private-addresses $TOKEN_FLAGS

# validator k (--genesis-validator only for k <= N0, and only until genesis is recorded)

f1r3node run --config-file "$STATE/conf/validator-$k.conf" \
--bootstrap "rnode://$BOOT_ID@127.0.0.1?protocol=40400&discovery=40404" \
--validator-private-key-path "$STATE/keys/validator-$k.pem" [--genesis-validator] \
--host 127.0.0.1 --no-upnp --allow-private-addresses $TOKEN_FLAGS

# observer
f1r3node run --config-file "$STATE/conf/observer.conf" \
--bootstrap "rnode://$BOOT_ID@127.0.0.1?protocol=40400&discovery=40404" \
--heartbeat-disabled --host 127.0.0.1 --no-upnp --allow-private-addresses
$TOKEN_FLAGS

TOKEN_FLAGS="--native-token-name F1R3CAP --native-token-symbol F1R3 --native-token-
decimals 8"

```

On the very first start the bootstrap's node id is not yet known; as in `shard.yml`, it bootstraps from itself, and `ign1t10n` learns the id from its `/api/status` before starting any other node.

7.2 Health

Every 10 s the supervisor reads `GET /api/status` from every node. The shard is Degraded when any node is not ready, when validators' last finalised blocks differ by more than 20, when the observer trails validator 1 by more than 20 blocks for more than 60 s, or when no block has been finalised for 120 s while `/api/pending-deploys` at validator 1 is non-empty. The status machine is Stopped $\rightarrow$ Starting(stage) $\rightarrow$ Running $\rightleftharpoons$ Degraded(reason), plus Resizing(step) and Failed(reason).

7.3 Restarts

A node that exits unexpectedly is restarted with exponential back-off (1, 2, 4, ..., 60 s); the others keep running. Five unexpected exits of one node within five minutes put the shard in Failed: the supervisor stops the whole shard, in the order below, rather than leave a bonded validator down (Principle 5), and posts a notification pointing at that node's log.

7.4 Stop

On SIGTERM: SIGTERM to the observer and wait up to 30 s; then to every validator at once and wait up to 60 s; then to the bootstrap and wait up to 30 s; SIGKILL whatever remains; exit 0. Validators stop together so that none is bonded and absent while the others continue.

7.5 Port conflicts at start

If a recorded port is taken at start, the shard goes to Failed naming the port and the holder (via `lssof`), and offers "Choose new ports" for the affected node. Moving any node other than the bootstrap needs no other change; moving the bootstrap rewrites every node's `--bootstrap`. No re-genesis is needed.

7.6 Sleep and wake

The nodes run under a launch agent, so App Nap does not apply. The supervisor holds a `PreventUserIdleSystemSleep` assertion only during provisioning and resizes.

8 Resizing the shard

A resize changes N to M with $2 \leq M \leq 10$. It is requested from the configuration panel or with `ctl resize`. Two modes exist.

8.1 In place (default)

The running shard keeps its chain, registry, balances and F1R3Gaze configuration. Validators are added at the lowest free slots and removed from the highest occupied slots; validator 1 is never removed. The resize is a list of per-slot steps recorded in `shard.toml` before the first is taken; each slot moves through the states below, and a restart of `ign1t10n` resumes each slot where it stopped.

Growing: slot k from absent to active.

1. **keyed**: allocate ports; generate the key, PEM and Keychain item; render `validator- $k$ .conf`.
2. **funded**: sign, with the faucet key, a transfer to validator k 's vault of the stake plus a fee reserve (1000 F1R3 + 1000 F1R3 by default), rendered with `gaze-wallet`'s transfer template; send it to validator 1; wait until the observer reports it finalised (`/api/deployment-finalization-status`) and the vault's balance (`/api/balance`) covers it.
3. **joining**: start the node (no `--genesis-validator`); wait for `/api/ready`, i.e. the last-finalised-state join has completed.
4. **bonding**: sign, with validator k 's own key, the bond deploy (the bundled template `bond.rho`, which is the repository's example with the amount set to the stake); send it to validator 1, a bonded validator, never to the joiner (Finding 5); wait until it is finalised and its result is `(true, _)`.
5. **active**: poll `GET /api/bond-status/{pk}` at the observer until `isBonded` is true, which happens at the next epoch boundary (at most `epoch-length = 10` blocks).

Slots grow one after another, so that every bond is submitted while the active set is known; the panel shows each step.

Shrinking: slot k from active to absent.

1. **retiring**: sign, with validator k 's key, the withdraw deploy (`PoS!("withdraw", deployerId, ret)`); send it to validator 1; wait until finalised with result `(true, _)`.
2. **left**: wait until the validator is no longer in the active set (`PoS!("getActiveValidators")`) by exploratory deploy at the observer), at the next epoch boundary. Only now stop the node (Principle 5).
3. **paid**: wait for the payout after `quarantine-length = 10` further blocks; sweep validator k 's vault, less the fee, back to the faucet with a transfer signed by validator k 's key.
4. **absent**: move the slot's data directory `nodes/validator- $k$ /`, its configuration and its PEM into a per-slot directory under `archive/<timestamp>/`; delete its Keychain item; free the slot.

The node leaves consensus at step 2; steps 3 and 4 are bookkeeping and do not block a further resize.

Failure. If a step fails (a deploy rejected, a timeout of five epochs’ worth of blocks without the expected state), the resize stops at that slot, the shard stays Running with the slots completed so far, and the panel offers “Retry” or “Undo”, where undo runs the opposite procedure on the slots already changed. A slot in joining whose bond never lands is stopped and archived, since it is not bonded.

8.2 As a new shard

“Start a new shard with M validators” is the reset of §13.2 with $N_0 := M$: the current shard is archived and provisioning runs again from S1. It is faster than growing by many slots and gives M genesis validators with equal stakes; it discards the chain. The panel says so.

8.3 Invariants

- $2 \leq N \leq 10$ at every step; a request outside the range is refused by the supervisor as well as the panel.
- Validator 1 is always active; F1R3Gaze’s validator setting never changes during a resize.
- At most one resize is in progress; start, stop and reset are refused while one runs, except “Stop”, which first finishes or undoes the current step.

9 User interface

9.1 Menu bar

A monochrome template image of the F1R3FLY.io firefly with a status dot: none for Running, amber for Starting, Resizing and Degraded, red for Failed, hollow for Stopped.

Item	Action
<i>Local shard: Running, 2 validators</i>	status line; submenu: block height, per-node readiness, observer lag, disk used
Open F1R3Gaze	<code>open -b io.f1r3fly.f1r3gaze</code>
Start / Stop shard	through the control socket
Configure... (Command-comma)	opens the configuration panel (§9.2)
Fund a wallet...	faucet transfer to a chosen F1R3Gaze wallet address
Copy endpoints	validator 1 and observer URLs, shard id
Show logs	reveals <code>~/Library/Logs/F1R3FLY/ign1t10n</code>
Reset shard...	§13.2
Uninstall...	§13.3
Quit ign1t10n	quits the menu bar only; the shard keeps running

9.2 The configuration panel

A standard settings window with two panes.

Shard.

- **Validators:** a stepper and slider, $2 \dots 10$, showing the current N ; changing it enables “Apply...”. Beside it, the estimated memory and disk for $M + 2$ nodes from the compiled footprint (§6.11), against this Mac’s physical memory and free disk. Above 50 % of physical

memory the estimate turns amber with a warning; above 80 % “Apply” asks for confirmation. Fault tolerance is shown from the measured table of Obligation 4, e.g. “finality continues with 1 validator stopped”, not from a formula.

- **Node table:** bootstrap, each validator slot and the observer, with base port, short public key, state (active, joining, bonding, retiring, down), last finalised block, and bonded stake.
- **Apply...**: a sheet offering “Resize the running shard (keeps everything; takes about one epoch per validator)” as the default and “Start a new shard with *M* validators (erases the local chain)”; the second requires typing the number again. While a resize runs, the pane shows its steps per slot, with “Retry” and “Undo” on failure.

General. “Run shard at login” (SMAppService register / unregister); “Use the local shard in F1R3Gaze” (the managed settings block); the ports of each node with “Choose new ports”; “Genesis amounts” shown read-only.

Everything the panel does goes through the control socket, so `ctl` can do the same and the panel never acts on the nodes directly.

9.3 First-run window

One window, F1R3FLY.io brand on a white surface (the white-background logo variant; yellow and sky only as rules and progress), listing S0–S9 with a spinner, check or error each, and an “Advanced” disclosure with the validator count (default 2). Errors name the stage, the cause and the log line.

10 Integration with F1R3Gaze

A F1R3Gaze configured by S8 resolves sites and reads at the observer at the *node* rung, has deploys quoted at the observer, signs them with the active wallet, sends them to validator 1, and follows them to finalisation through the observer’s `/ws/events`. Resizing does not touch F1R3Gaze’s configuration.

Work package 1 (G1: porcelain wallet commands). `f1r3gaze wallet active --porcelain` prints the active address or exits 3; `f1r3gaze wallet new LABEL --porcelain` prints only the new address. Both are safe while a window is open. Estimate: 0.5 engineer-week.

Work package 2 (G2: balances without Embers). When `embers_api` is unset, balances come from the first observer’s `GET /api/balance/{address}`. Estimate: 0.5 engineer-week.

Work package 3 (G3: a settings drop-in directory). F1R3Gaze parses `settings.d/*.conf` after `settings.conf`, in lexical order, last assignment winning. Estimate: 0.25 engineer-week.

Work package 4 (G4: several validators (optional)). Let validator take a list and send each deploy to the first that accepts it, so that F1R3Gaze survives validator 1 being restarted. Not required for version 1; Decision 5. Estimate: 0.5 engineer-week.

`ign1t10n` depends on `gaze-shard` and `gaze-wallet` at the revision of the F1R3Gaze release it bundles; the two crates SHOULD expose `deploy` construction, signing and `render_transfer` as public API without the browser’s broker (a small refactor inside G1’s estimate).

11 Required changes to F1R3Node-Rust

Work package 5 (N1: loopback binding). Add `protocol-server.bind-address` and `peers-discovery.bind-address` (default `0.0.0.0`) and use them where the transport receiver and the Kademlia server bind. `ign1t10n` sets both, and `api-server.host`, to `127.0.0.1`. Estimate: 1 engineer-week.

Work package 6 (N2: a self-contained macOS build). A vendored-openssl feature in `crypto`; a CI job on `macos-14` building both architectures, running unit tests on `arm64`, and failing if `otool -L` shows a library outside the system. Estimate: 1 engineer-week.

Work package 7 (N3: the loopback shard test). An integration test (and `docker/-free` script) running, on one host with every node on `127.0.0.1` and distinct ports: a bootstrap, N genesis validators for $N \in \{2, 4, 10\}$, and an observer; asserting genesis, finalisation of a deploy sent to validator 1 as seen at the observer, registry and exploratory reads at the observer; then growing by one validator (fund, join, bond, epoch boundary) and shrinking by one (withdraw, leave, payout), with deploys finalising throughout. Run on Linux in CI and on macOS in N2's job. Fix what it exposes. Discharges Obligations 1 and 2. Estimate: 2 engineer-weeks.

Work package 8 (N4: storage-format metadata). A storage-format integer in `--version` and `/api/status`, changed whenever a release cannot open the previous release's data directory. Estimate: 0.25 engineer-week.

12 Security model

Threat	Protection
Reuse of public development keys	All keys generated at S3 or at resize; the release pipeline greps every artifact for the public keys in <code>docker/.env.example</code> and fails on a match.
Another user on the Mac	State directory <code>0700</code> , PEMs <code>0600</code> ; no key in any argument vector; each PEM password only in the Keychain and in its own node's environment; faucet key never given to a node; control socket checks the peer uid.
Hosts on the same network	Every listener on loopback (N1 and <code>api-server.host</code>).
Pages in other browsers	A stock browser can reach the loopback HTTP APIs. It can submit deploys signed with its own keys and paid from its own funds; it cannot sign as any local key. Accepted for a development shard; Decision 7.
Pages inside F1R3Gaze	F1R3Gaze's own model: capabilities, consent prompts, the system-name allow-list refusing <code>rho:rchain:deployerId</code> .
Tampered artifact	Developer ID signatures on every Mach-O and on the archive; notarisation; hardened runtime with no entitlements; a GPG-signed SHA256SUMS.
Malware running as the user	Not defended; the hardened runtime at least prevents attaching a debugger to a node to read a decrypted key.

13 Upgrade, reset and uninstall

13.1 Upgrade

A newer archive replaces both bundles. The supervisor, restarted by `launchd`, compares the new `f1r3node`'s storage format (N4) with the one recorded at genesis. If compatible, it restarts every

node on its data, finishing any interrupted resize. If not, it does not start; the panel offers “Start a new shard” with the current N , which archives the shard and re-provisions from S3, funding the wallet again. A resize in progress blocks an upgrade’s restart until the current step completes.

13.2 Reset

“Reset shard...” stops the shard, moves `nodes/`, `genesis/`, `conf/` and `keys/` into a timestamped directory under `archive/`, deletes the Keychain items, and re-runs provisioning from S1 with the chosen N_0 (the current N by default). The dialog says that every deploy, registry entry and balance on the local shard will be gone, and that F1R3Gaze wallets are untouched and the active one will be funded again.

13.3 Uninstall

Stop the shard; unregister the agent; after a second confirmation remove the state directory (optionally keeping `archive/`), the logs and every Keychain item of service `io.f1r3fly.ign1t10n`; remove F1R3Gaze’s managed block or drop-in; move `ign1t10n.app` to the Trash; `pkgutil --forget io.f1r3fly.ign1t10n.pkg`. F1R3Gaze, its profile and its wallets are left alone.

14 Build and release pipeline

A repository `F1R3FLY-io/ign1t10n` holds the crate, templates, `packaging/macos/` and `versions.toml`, which pins the F1R3Node-Rust revision and the F1R3Gaze release. A tag `vX.Y.Z` runs the release workflow.

Job	Runner	Steps
node	macos-14	F1R3Node-Rust at the pinned revision, its pinned toolchain, protobuf from Homebrew; <code>-p node --release --features crypto/vendored-openssl</code> for both targets; <code>lipo</code> ; <code>otool -L gate</code>
gaze	ubuntu	download the pinned F1R3Gaze DMG, SHA256SUMS and signature; verify
ign1t10n	macos-14	build for both targets; unit tests (PEM round-trip, resize state machine against a mock node); <code>lipo</code>
assemble	macos-14	verify the F1R3Gaze bundle’s signature; assemble <code>ign1t10n.app</code> ; sign inside-out; <code>pkgbuild</code> both components; <code>productbuild</code> ; sign with the installer certificate; notarise; staple; <code>pkgutil --check-signature</code> ; <code>spctl -a -t install</code>
smoke	macos-14	install; headless provisioning (<code>ctl provision --non-interactive</code> , which launches the supervisor directly instead of S6); acceptance checks A2–A6, A11, A12
publish	ubuntu	SHA256SUMS, GPG signature; draft GitHub release

```
cs() { codesign --force --timestamp --options runtime \
    --entitlements packaging/macos/entitlements.plist --sign "$APP_ID" "$@"; }
cs ign1t10n.app/Contents/Helpers/f1r3node
cs ign1t10n.app/Contents/MacOS/ign1t10n
cs ign1t10n.app
pkgbuild --component ign1t10n.app --install-location /Applications \
    --scripts packaging/macos/scripts --identifier io.f1r3fly.ign1t10n.pkg ...
pkgbuild --component F1R3Gaze.app --install-location /Applications \
```

```

--identifier io.f1r3fly.f1r3gaze.pkg ...    # the release bundle, not re-
signed
productbuild --distribution Distribution.xml --package-path . unsigned.pkg
productsign --sign "$INSTALLER_ID" --timestamp unsigned.pkg ign1t10n-$V-macos-
universal.pkg
xcrun notarytool submit ... --wait && xcrun stapler staple ign1t10n-$V-macos-universal.
pkg

```

Secrets: F1R3Gaze's (MACOS_CERT_P12, MACOS_CERT_PASSWORD, MACOS_SIGN_IDENTITY, APPLE_ID, APPLE_TEAM_ID, APPLE_APP_PASSWORD), plus MACOS_INSTALLER_CERT_P12, MACOS_INSTALLER_CERT_PASSWORD, MACOS_INSTALLER_SIGN_IDENTITY for the Developer ID Installer certificate, and GPG_PRIVATE_KEY, GPG_PASSPHRASE. The entitlements file is empty.

15 Testing and acceptance

A1 Clean install.

On a fresh user account, the archive (plus one background-item approval) ends with F1R3Gaze open and `ctl status` reporting Running, 2 validators, within five minutes.

A2 Round trip through the browser.

`f1r3gaze --headless` on a test page that inserts into the registry is signed by the funded wallet, sent to validator 1, reported finalised, and read back at the observer.

A3 Loopback only.

`lsof -nP -iTCP -sTCP:LISTEN` lists no `f1r3node` socket on an address other than `127.0.0.1`.

A4 No foreign libraries.

`otool -L` on every Mach-O lists only system paths.

A5 No shipped secrets.

No public key in `shard.toml` appears in `docker/.env.example`; PEMs are 0600; `ps -Eww` by another user reveals no key or password.

A6 Persistence.

After a reboot, the shard returns to Running with the same genesis hash, the same N , and a height no lower than before.

A7 Port conflict.

With `docker/shard.yml` running (which occupies the default plan), provisioning allocates other bases and A2 passes.

A8 Crash recovery.

`kill -9` of any one node leads back to Running; six rapid kills of one validator stop the whole shard in Failed.

A9 Upgrade.

v to $v+1$ keeps the chain and N with a compatible storage format; with an incompatible one the shard is archived only after consent.

A10 Uninstall.

No state, logs, Keychain items or agent remain; F1R3Gaze and its wallets are untouched.

A11 Grow in place.

From $N = 2$, `ctl resize 5`: validators 3–5 reach active, `/api/bond-status` is true for each,

and a deploy sent during the resize and one after it both finalise. The chain's height never decreases.

A12 Shrink in place.

From $N = 5$, `ctl resize 3`: validators 5 and 4 withdraw, leave the active set before their processes stop, are paid out, and are swept to the faucet; deploys finalise throughout; validator 1 is untouched.

A13 New shard at the maximum.

"Start a new shard with 10 validators" produces a genesis with ten bonds and A2 passes.

A14 Bounds and interruption.

`ctl resize 1` and `ctl resize 11` are refused; killing the supervisor during each growing and shrinking step and restarting it completes the resize.

The matrix is macOS 13, 14, 15, 26 and the current release on Apple silicon, and 13 and 15 on Intel while x86_64 is shipped; A11–A13 also run on the smallest supported memory configuration to calibrate the panel's warnings.

16 Work packages and schedule

WP	Deliverable	Est.
N1	node: configurable bind address for transport and discovery	1
N2	node: vendored OpenSSL; macOS CI build with <code>otool gate</code>	1
N3	node: loopback shard test, $N \in \{2, 4, 10\}$, grow and shrink; fixes	2
N4	node: storage-format number	0.25
G1–G3	F1R3Gaze: porcelain wallet commands; balances from the node; <code>settings.d</code> ; public deploy and transfer API	1.25
H1	ign1t10n: manifest, provisioning, configuration rendering, keys and PEMs, Keychain	2
H2	ign1t10n supervise: multi-node process management, genesis ceremony, health, restarts, shutdown, control socket, <code>ctl</code>	2.5
H3	menu bar, first-run window, <code>SMApService</code> , notifications, brand assets	1.5
H4	resize engine: per-slot state machine, faucet transfers, bond and withdraw deploys, sweep, undo	2.5
H5	configuration panel: shard and general panes, footprint estimates, node table, apply sheet	1.5
H6	reset, upgrade compatibility, uninstall, "Fund a wallet"	1
R1	release workflow	1.5
T1	acceptance suite, footprint and tolerance measurements, pre-release matrix	2
Total		20

N3 is on the critical path: it proves the topology and the resize procedure before H2 and H4 depend on them. With two engineers, one on the node, the resize engine and the pipeline and one on the rest of ign1t10n, the work fits in ten to eleven calendar weeks.

17 Obligations and decisions

Obligation 1 (The loopback shard works). Before H2 is merged, N3 must show the `shard.yml` topology running with every node on `127.0.0.1` and distinct ports, for $N = 2$ and $N = 10$.

Obligation 2 (Resizing works). Before H4 is merged, N3 must show one validator joining, bonding and becoming active, and one withdrawing, leaving and being paid out, with the shard configuration's epoch and quarantine lengths, and with deploys finalising throughout.

Obligation 3 (Footprint). T1 measures, at the pinned node version, each role's resident memory and disk after one hour of a light deploy load, for $N = 2$ and $N = 10$; the figures are compiled into `ign1t10n` as $M_{\min}$, $D_{\min}$ and the panel's estimates, and re-measured whenever the pinned node changes.

Obligation 4 (Measured fault tolerance). For each N from 2 to 10, T1 measures how many validators may be stopped while blocks continue to finalise, under `fault-tolerance-threshold = 0.1`, and the panel shows that table. With $N = 2$ the expectation is none; the panel says so plainly.

Obligation 5 (PEM compatibility). The PBES2 parameters of §5 must be accepted by the OpenSSL that N2 vendors; the round-trip test in H1 proves it in CI.

Open decision 1 (Fold the bootstrap into validator 1). The bootstrap costs one node's memory and adds a process that proposes nothing. A bonded ceremony master exists already (standalone mode is one), so validator 1 could take the role, leaving exactly N validators plus the observer. Proposed: keep the tested `shard.yml` shape for version 1 and revisit once N3 is green.

Open decision 2 (Intel). Universal binaries, as F1R3Gaze ships, or Apple silicon only, which also matters more now that ten validators is a supported configuration.

Open decision 3 (Minimum macOS). 13.0, for `SMAccessService`.

Open decision 4 (Genesis amounts and stakes). One million F1R3 to the browser's wallet, 1 000 F1R3 of fees and a stake of 1 000 to each validator, the rest to the faucet; or stakes chosen per validator in the panel.

Open decision 5 (Where deploys go). Always validator 1 (proposed), or F1R3Gaze rotating across validators with G4.

Open decision 6 (Heartbeat and loop timing). Keep `default.conf`'s timings (casper loop 750 ms, heartbeat on) or tune them for a laptop once T1 has measured finalisation latency and idle CPU at $N = 10$.

Open decision 7 (Cross-origin requests). Whether the node should refuse HTTP requests carrying a foreign `Origin` header, so that pages in other browsers cannot use the local shard's API.

Open decision 8 (Embers). Whether a later version bundles Embers, so that transfers, not just balances, work in F1R3Gaze against the local shard.

A Configuration templates

`conf/common.conf`

```

# Rendered by ignit10n. The node loads its built-in defaults.conf first;
# the values below are docker/conf/default.conf's, generalised to loopback.
protocol-server {
    network-id = "ignit10n-{{NETWORK_SUFFIX}}"
    bind-address = "127.0.0.1"           # N1; omitted until N1 lands
    allow-private-addresses = true
    no-upnp = true
}
peers-discovery.bind-address = "127.0.0.1"
api-server.host = "127.0.0.1"
casper {
    shard-name = "root"
    fault-tolerance-threshold = 0.1
    casper-loop-interval = 750 ms
    requested-blocks-timeout = 5 seconds
    fork-choice-stale-threshold = 1 minutes
    fork-choice-check-if-stale-interval = 20 seconds
    max-number-of-parents = 100
    enable-mergeable-channel-gc = true
    heartbeat { enabled = true }
    genesis-ceremony {
        required-signatures = {{NO}}
        approve-interval = 10 seconds
        approve-duration = 1 minutes
    }
    genesis-block-data {
        genesis-data-dir = "{{STATE}}/genesis"
        bonds-file = "{{STATE}}/genesis/bonds.txt"
        wallets-file = "{{STATE}}/genesis/wallets.txt"
        number-of-active-validators = 10
        epoch-length = 10
        quarantine-length = 10
    }
}
logging { format = "json", sink = "file", file { rotation = "daily", retention = 7 } }
metrics { prometheus = true }

```

conf/bootstrap.conf

```

include "common.conf"
casper.genesis-ceremony.ceremony-master-mode = true
protocol-server.port = 40400
peers-discovery.port = 40404
api-server { port-grpc-external = 40401, port-grpc-internal = 40402,
             port-http = 40403, port-admin-http = 40405 }
storage.data-dir = "{{STATE}}/nodes/bootstrap"
tls { certificate-path = "{{STATE}}/nodes/bootstrap/node.certificate.pem",
       key-path = "{{STATE}}/nodes/bootstrap/node.key.pem" }

```

conf/validator-*{k}*.conf

```
include "common.conf"
protocol-server.port = {{BASE}}           # 40410 for k = 1, 40420 for k = 2, ...
peers-discovery.port = {{BASE+4}}
api-server { port-grpc-external = {{BASE+1}}, port-grpc-internal = {{BASE+2}},
              port-http = {{BASE+3}}, port-admin-http = {{BASE+5}} }
storage.data-dir = "{{STATE}}/nodes/validator-{{k}}"
tls { certificate-path = "{{STATE}}/nodes/validator-{{k}}/node.certificate.pem",
      key-path = "{{STATE}}/nodes/validator-{{k}}/node.key.pem" }
```

conf/observer.conf

```
include "common.conf"
casper.heartbeat.enabled = false
protocol-server.port = 40450
peers-discovery.port = 40454
api-server { port-grpc-external = 40451, port-grpc-internal = 40452,
              port-http = 40453, port-admin-http = 40455 }
storage.data-dir = "{{STATE}}/nodes/observer"
tls { certificate-path = "{{STATE}}/nodes/observer/node.certificate.pem",
      key-path = "{{STATE}}/nodes/observer/node.key.pem" }
```

{{STATE}} contains a space; every path is quoted. include is resolved relative to the including file, as in docker/conf/readonly-remote.conf.

B The manifest

```
# ~/Library/Application Support/F1R3FLY/ign1t10n/shard.toml
schema = 2

[shard]
id = "root"
network_id = "ign1t10n-3f9c1a07"
genesis_hash = "a41c...e2"
genesis_validators = 2           # NO
genesis_node_version = "0.4.46"
storage_format = 7              # from N4
created = 2026-10-14T09:12:44Z

[bootstrap]
public_key = "04e2...0b"
node_id = "5b0e...c1"
base_port = 40400

[observer]
base_port = 40450

[faucet]
address = "1111...Fq"
```

```

[wallet]
funded = "1111...Zr"
gaze_integration = true

[[validators]]
slot = 1
public_key = "04a1...9e"
address = "1111...Qk"
base_port = 40410
state = "active"                # absent/keyed/funded/joining/bonding/active/
                                retiring/left/paid
genesis = true

[[validators]]
slot = 2
public_key = "04c7...31"
address = "1111...Hm"
base_port = 40420
state = "active"
genesis = true

[resize]                        # present only while a resize runs
target = 4
mode = "in-place"
started = 2026-10-20T16:02:10Z
steps = [ { slot = 3, state = "bonding", deploy = "3045...aa" },
           { slot = 4, state = "absent" } ]

[stages]                        # provisioning progress, S0..S9
preflight = true
ports = true
wallet = true
keys = true
genesis_files = true
configs = true
agent = true
genesis = true
gaze = true
opened = true

```